

ITA0605 - Machine Learning

List of Lab Exercises

7. Write a program to implement Logistic Regression (LR) algorithm in python.

Aim:

To implement the **Logistic Regression (LR)** algorithm in Python for binary classification and evaluate its accuracy.

Algorithm:

1. Import the required Python libraries.
2. Load the dataset.
3. Select the input features (**X**) and target (**y**).
4. Split the dataset into training and testing sets.
5. Create a Logistic Regression model.
6. Train the model using the training data.
7. Predict the class labels for the test data.
8. Calculate and display the accuracy of the model.

Program Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data

y = data.target

X = X[y != 2]
```

```
y = y[y != 2]

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Actual Labels:", y_test)

print("Predicted Labels:", y_pred)

print("Accuracy:", accuracy)
```

Output:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Use only two classes (Binary Classification)
X = X[y != 2]
y = y[y != 2]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual Labels:", y_test)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy)
```

... Actual Labels: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Predicted Labels: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Accuracy: 1.0

Dataset 2:

Program Code:

```
import pandas as pd

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import confusion_matrix, accuracy_score
```

```
data = [
    [8, 780, 5, 1],
    [7, 760, 4, 1],
    [6, 730, 5, 1],
    [4, 620, 8, 0],
    [3, 590, 9, 0],
    [9, 810, 4, 1],
    [5, 650, 7, 0],
    [8, 770, 5, 1],
    [4, 610, 8, 0],
```

```
[7, 750, 6, 1]  
]
```

```
columns = [  
    'Income',  
    'Credit Score',  
    'Loan Amount',  
    'Loan Approved'  
]
```

```
df = pd.DataFrame(data, columns=columns)
```

```
print("Original Dataset:")  
print(df)
```

```
X = df[['Income', 'Credit Score', 'Loan Amount']]  
y = df['Loan Approved']
```

```
model = GaussianNB()
```

```
model.fit(X, y)
```

```
print("\nNaive Bayes Model Training Completed Successfully.")
```

```
y_pred = model.predict(X)
```

```
cm = confusion_matrix(y, y_pred)
```

```
print("\nConfusion Matrix:")  
print(cm)
```

```
accuracy = accuracy_score(y, y_pred)
```

```
print("\nAccuracy:")
```

```
print(accuracy)
```

```
print("\nAccuracy Percentage:")
```

```
print(accuracy * 100, "%")
```

```
new_sample = pd.DataFrame(  
    [[7, 760, 5]],  
    columns=['Income', 'Credit Score', 'Loan Amount']  
)
```

```
prediction = model.predict(new_sample)
```

```
print("\n=====")
```

```
print("Loan Approval Prediction")
```

```
print("=====")
```

```
print("\nNew Sample:")
```

```
print(new_sample)
```

```
print("\nPredicted Result:")
```

```
if prediction[0] == 1:
```

```
    print("Approved")
```

```
else:
```

```
    print("Rejected")
```

```
print("=====")
```

Output:

```
Original Dataset:
***   Income  Credit Score  Loan Amount  Loan Approved
0      8      780         5           1
1      7      760         4           1
2      6      730         5           1
3      4      620         8           0
4      3      590         9           0
5      9      810         4           1
6      5      650         7           0
7      8      770         5           1
8      4      610         8           0
9      7      750         6           1

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy:
1.0

Accuracy Percentage:
100.0 %

=====
Loan Approval Prediction
=====

New Sample:
   Income  Credit Score  Loan Amount
0       7      760         5

Predicted Result:
Approved
-----
```

Dataset 3:

Program Code:

```
import pandas as pd

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

data = [
    [39.1, 112, 94, 1],
    [38.8, 109, 95, 1],
    [37.0, 78, 99, 0],
    [39.3, 115, 93, 1],
    [36.8, 74, 98, 0],
    [38.7, 110, 94, 1],
    [37.1, 80, 98, 0],
```

```
[39.0, 113, 92, 1],  
[36.9, 76, 99, 0],  
[38.9, 111, 93, 1]  
]
```

```
columns = [  
    'Temperature',  
    'Heart Rate',  
    'Oxygen Level',  
    'Disease'  
]
```

```
df = pd.DataFrame(data, columns=columns)
```

```
print("Original Dataset:")  
print(df)
```

```
X = df[['Temperature', 'Heart Rate', 'Oxygen Level']]  
y = df['Disease']
```

```
model = GaussianNB()
```

```
model.fit(X, y)
```

```
print("\nNaive Bayes Model Training Completed Successfully.")
```

```
y_pred = model.predict(X)
```

```
cm = confusion_matrix(y, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
accuracy = accuracy_score(y, y_pred)
```

```
print("\nAccuracy:")
```

```
print(accuracy)
```

```
print("\nAccuracy Percentage:")
```

```
print(accuracy * 100, "%")
```

```
new_sample = pd.DataFrame(  
    [[38.8, 110, 94]],  
    columns=['Temperature', 'Heart Rate', 'Oxygen Level']  
)
```

```
prediction = model.predict(new_sample)
```

```
print("\n=====")
```

```
print("Disease Diagnosis Prediction")
```

```
print("=====")
```

```
print("\nNew Sample:")
```

```
print(new_sample)
```

```
print("\nPredicted Result:")
```

```
if prediction[0] == 1:
```

```
    print("Disease Present (1)")
```

```
else:
```

```
    print("Disease Absent (0)")
```

```
print("=====")
```


Output:

```
Original Dataset:
  Temperature  Heart Rate  Oxygen Level  Disease
...  0         39.1       112           94        1
  1         38.8       109           95        1
  2         37.0        78           99        0
  3         39.3       115           93        1
  4         36.8        74           98        0
  5         38.7       110           94        1
  6         37.1        80           98        0
  7         39.0       113           92        1
  8         36.9        76           99        0
  9         38.9       111           93        1

Naive Bayes Model Training Completed Successfully.

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy:
1.0

Accuracy Percentage:
100.0 %

=====
Disease Diagnosis Prediction
=====

New Sample:
  Temperature  Heart Rate  Oxygen Level
0         38.8       110           94

Predicted Result:
Disease Present (1)
=====
```

Dataset 4:

Program Code:

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X = np.array([
    [10, 95, 50],
    [9, 92, 48],
    [8, 90, 45],
    [4, 70, 25],
    [3, 65, 20],
    [11, 96, 52],
    [5, 75, 30],
    [8, 89, 44],
    [2, 60, 18],
    [7, 87, 40]
])
```

```
y = np.array([
    1,
    1,
    1,
    0,
    0,
    1,
    0,
    1,
    0,
    1
])
```

```
model = GaussianNB()
```

```
model.fit(X, y)
```

```
print("Model Training Completed Successfully.")
```

```
y_pred = model.predict(X)
```

```
cm = confusion_matrix(y, y_pred)
```

```
accuracy = accuracy_score(y, y_pred)
```

```
new_sample = np.array([
    [8, 91, 46]
])
```

```
prediction = model.predict(new_sample)
```

```
print("\n-----")
print("Naive Bayes Prediction")
print("-----")
```

```
print("\nNew Sample:")
print("Experience      :", 8)
print("Performance Score:", 91)
print("Training Hours   :", 46)
```

```
print("\nConfusion Matrix:")
print(cm)
```

```
print("\nAccuracy:", accuracy * 100, "%")
```

```
print("\nPredicted Result:")
```

```
if prediction[0] == 1:
```

```
    print("1 - Promoted")
else:
    print("0 - Not Promoted")

print("-----")
```

Output:

```
--- Model Training Completed Successfully.

-----
Naive Bayes Prediction
-----

New Sample:
Experience      : 8
Performance Score: 91
Training Hours  : 46

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy: 100.0 %

Predicted Result:
1 - Promoted
-----
```

Dataset 5:

Program Code:

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
X = np.array([
    [22, 3, 30],
    [25, 4, 35],
    [28, 5, 45],
    [32, 7, 60],
    [35, 8, 65],
    [40, 9, 75],
    [45, 10, 80],
    [30, 6, 55],
    [24, 4, 40],
    [38, 9, 70]
])
```

```
y = np.array([
    0,
    0,
```

```
0,
1,
1,
1,
1,
1,
0,
1
])
```

```
model = GaussianNB()
```

```
model.fit(X, y)
```

```
print("Model Training Completed Successfully.")
```

```
y_pred = model.predict(X)
```

```
cm = confusion_matrix(y, y_pred)
```

```
accuracy = accuracy_score(y, y_pred)
```

```
new_sample = np.array([
    [34, 8, 68]
])
```

```
prediction = model.predict(new_sample)
```

```
print("\n-----")
print("Naive Bayes Prediction")
print("-----")
```

```
print("\nNew Sample:")
print("Age      :", 34)
print("Annual Income  :", 8, "Lakhs")
print("Spending Score :", 68)
```

```
print("\nConfusion Matrix:")
print(cm)
```

```
print("\nAccuracy:", accuracy * 100, "%")
```

```
print("\nPredicted Result:")
```

```
if prediction[0] == 1:
```

```
    print("1 - Purchased")
else:
    print("0 - Not Purchased")

print("-----")
```

Output:

```
... Model Training Completed Successfully.

-----
Naive Bayes Prediction
-----

New Sample:
Age          : 34
Annual Income : 8 Lakhs
Spending Score : 68

Confusion Matrix:
[[4 0]
 [0 6]]

Accuracy: 100.0 %

Predicted Result:
1 - Purchased
-----
```

Result:

The **Logistic Regression algorithm** was successfully implemented in Python. The model classified the data correctly and achieved **100% accuracy (1.0)** on the test dataset.